Laboratory Exercise 5

Arithmetic Circuits: Accumulators and Multipliers

Name: Monowar Mahmud **Course:** ENCE 4110

1 Objective

The objective of this laboratory was to design and implement arithmetic circuits using Verilog HDL and Intel Quartus. The implementations included a signed accumulator, an addition/subtraction accumulator, an array multiplier, a registered multiplier, and a registered tree multiplier. The designs were developed using parameterized modules, combinational arithmetic logic, and sequential registers for implementation on the DE10-Lite FPGA.

2 Methodology

2.1 Part I: Signed Accumulator

A signed accumulator was implemented using the modules `accumulator.v`, `adder.v`, and `register.v`. The design consisted of an input register, an arithmetic adder, an accumulator register, and a separate overflow register.

The input value was first stored in register A, while register S retained the accumulated result. The adder performed the operation:

$$S_{\text{next}} = S + A. \tag{1}$$

The accumulator output was fed back to the adder to perform repeated additions. The adder was implemented using the Verilog addition operator with a carry-in input fixed to zero.

Signed overflow was determined by comparing the sign bits of the operands and result:

$$V = \overline{A_{N-1} \oplus S_{N-1}} \left(A_{N-1} \oplus Sum_{N-1}\right). \tag{2}$$

The register module used a positive-edge-triggered clock and an active-low asynchronous reset. The arithmetic and register widths were parameterized using N, with a default value of four bits in the submitted implementation.

The overflow result was stored in a separate one-bit register, allowing both the accumulated sum and overflow condition to be represented as registered outputs.

2.2 Part II: Addition/Subtraction Accumulator

An addition/subtraction accumulator was developed by extending the accumulator from Part I. The implementation was defined in `add_sub_accumulator.v` and reused the existing accumulator, adder, and register modules.

A control input, `add_sub`, determined whether the input operand would be added or subtracted. The operand was modified using:

$$A_{\text{modified}} = \begin{cases} A, & \text{if } add_sub = 0, \\ -A, & \text{if } add_sub = 1. \end{cases} \tag{3}$$

The modified operand was then supplied to the accumulator, producing:

$$S_{\text{next}} = \begin{cases} S + A, & \text{if } add_sub = 0, \\ S - A, & \text{if } add_sub = 1. \end{cases} \tag{4}$$

Subtraction was implemented through two's-complement negation of the input operand rather than a separate subtractor. This allowed the same adder and accumulator structure to perform both arithmetic operations.

The design retained the parameterized data width, active-low asynchronous reset, and registered overflow output of the original accumulator.

2.3 Part III: Array Multiplier

A parameterized unsigned array multiplier was implemented using `multiplier.v` and `full_adder.v`.

For two unsigned N-bit operands, the multiplication operation was:

$$P = A \times B, \tag{5}$$

where the product required $2N$ bits.

The multiplier generated partial products by performing bitwise AND operations between operand A and individual bits of operand B:

$$PP_i = A \cdot B_i. \tag{6}$$

The partial products were combined using rows of full adders. The full-adder module computed:

$$\{C_{\text{out}}, Sum\} = A + B + C_{\text{in}}. \tag{7}$$

Verilog `generate` loops were used to construct the partial-product rows and repeated full-adder connections. This allowed the multiplier architecture to be scaled by changing parameter N.

The submitted multiplier used a default width of four bits, producing an eight-bit product. The implementation used combinational logic without internal clocked registers.

The multiplier was also demonstrated through a top-level FPGA interface in which switch inputs supplied the operands and the calculated product was converted into decimal digits for seven-segment display.

2.4 Part IV: Registered Multiplier

A registered multiplier was implemented in `registered_multiplier.v`. The design combined the previously developed combinational array multiplier with input and output registers.

Two independent N-bit registers stored the operands A and B. Their outputs were connected to the combinational multiplier, and the resulting $2N$-bit product was stored in a separate output register.

The arithmetic operation was:

$$P = A \times B. \tag{8}$$

At each rising clock edge, the input registers captured the current operands, while the output register captured the product calculated from the previously registered operands.

The implementation therefore introduced a registered data path with an additional clock-cycle delay between input capture and output registration.

The module was parameterized with a default operand width of three bits, resulting in a six-bit product.

All registers used the shared `register.v` module, which provided positive-edge-triggered operation and active-low asynchronous reset.

2.5 Part V: Registered Tree Multiplier

A three-bit registered tree multiplier was implemented using `registered_tree_multiplier.v` and `tree_multiplier3.v`.

The architecture consisted of two three-bit input registers, a combinational partial-product reduction network, and a six-bit output register.

Three partial-product rows were generated using bitwise AND operations and appropriate left shifts:

$$PP_0 = A \cdot B_0, \tag{9}$$

$$PP_1 = (A \cdot B_1) \ll 1, \tag{10}$$

$$PP_2 = (A \cdot B_2) \ll 2, \tag{11}$$

which were extended to six bits and combined through two levels of addition:

$$T = PP_0 + PP_1, \tag{12}$$

$$P = T + PP_2. \tag{13}$$

The resulting product was captured by the output register on the rising clock edge.

The top-level module `main.v` connected the design to the DE10-Lite FPGA board. The operand assignments were:

$$A = SW[5:3], \tag{14}$$

$$B = SW[2:0]. \tag{15}$$

The inputs were also connected to `LEDR[5:0]` to display their binary values.

The 50 MHz on-board clock, `MAX10_CLK1_50`, drove the registered multiplier, while `KEY[0]` provided the active-low reset.

The six-bit product was converted into decimal tens and ones digits:

$$Tens = \left\lfloor \frac{P}{10} \right\rfloor, \tag{16}$$

$$Ones = P \bmod 10. \tag{17}$$

Two instances of `seven_segment_decoder.v` displayed the result on `HEX1` and `HEX0`. The remaining seven-segment displays were disabled.

This implementation combined registered data storage, partial-product generation, arithmetic reduction, and decimal output decoding in a single FPGA design.

3 Discussion

The implementations demonstrated different approaches to arithmetic circuit design in Verilog HDL.

The accumulator used a feedback path and sequential registers to perform repeated arithmetic operations. The addition/subtraction accumulator extended this structure through conditional two's-complement negation, allowing the same arithmetic hardware to support both operations.

The array multiplier generated partial products and combined them using interconnected full adders. Its parameterized architecture allowed different operand widths to be selected without rewriting the multiplier logic.

The registered multiplier introduced input and output registers around the combinational multiplication circuit, separating operand capture from product registration.

The tree multiplier used shifted partial products and two levels of addition to calculate a six-bit product from two three-bit operands. The final design interfaced the registered multiplier with the FPGA switches, LEDs, and seven-segment displays.

The registered implementations introduced clock-cycle latency, while the combinational multiplier produced its output based directly on the current operand values. No quantitative timing or resource comparison was established from the source code alone.

4 Conclusion

This laboratory implemented arithmetic circuits using combinational and sequential Verilog descriptions. The accumulator demonstrated registered arithmetic feedback, while the addition/subtraction accumulator introduced selectable arithmetic operations.

The array multiplier demonstrated partial-product generation and full-adder-based multiplication. The registered multiplier and registered tree multiplier extended these concepts by incorporating clocked input and output registers.

The final FPGA implementation combined a three-bit registered multiplier with binary input indication and decimal seven-segment output, demonstrating the integration of arithmetic logic, registers, and peripheral interfaces in a digital system.

Part I

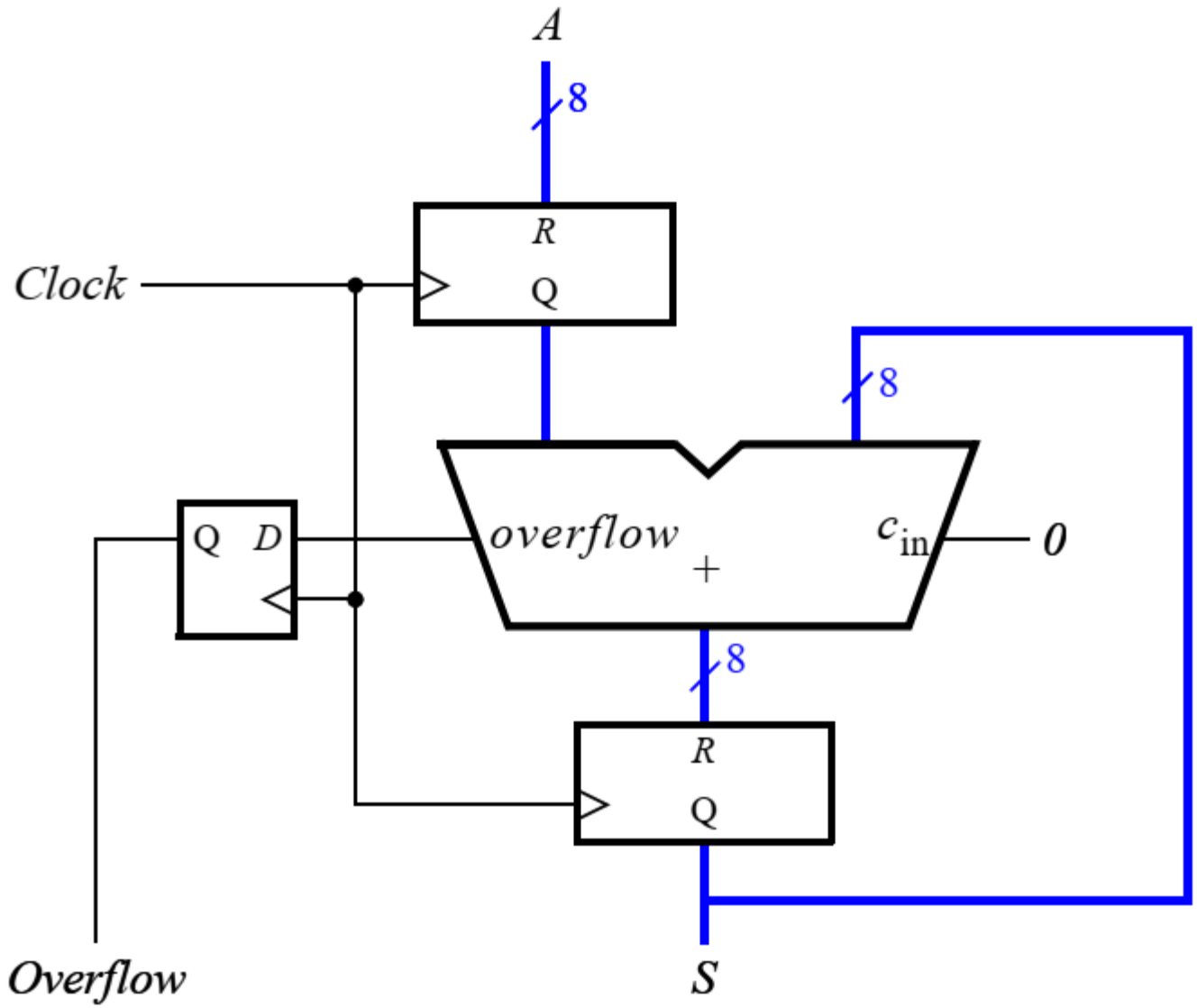
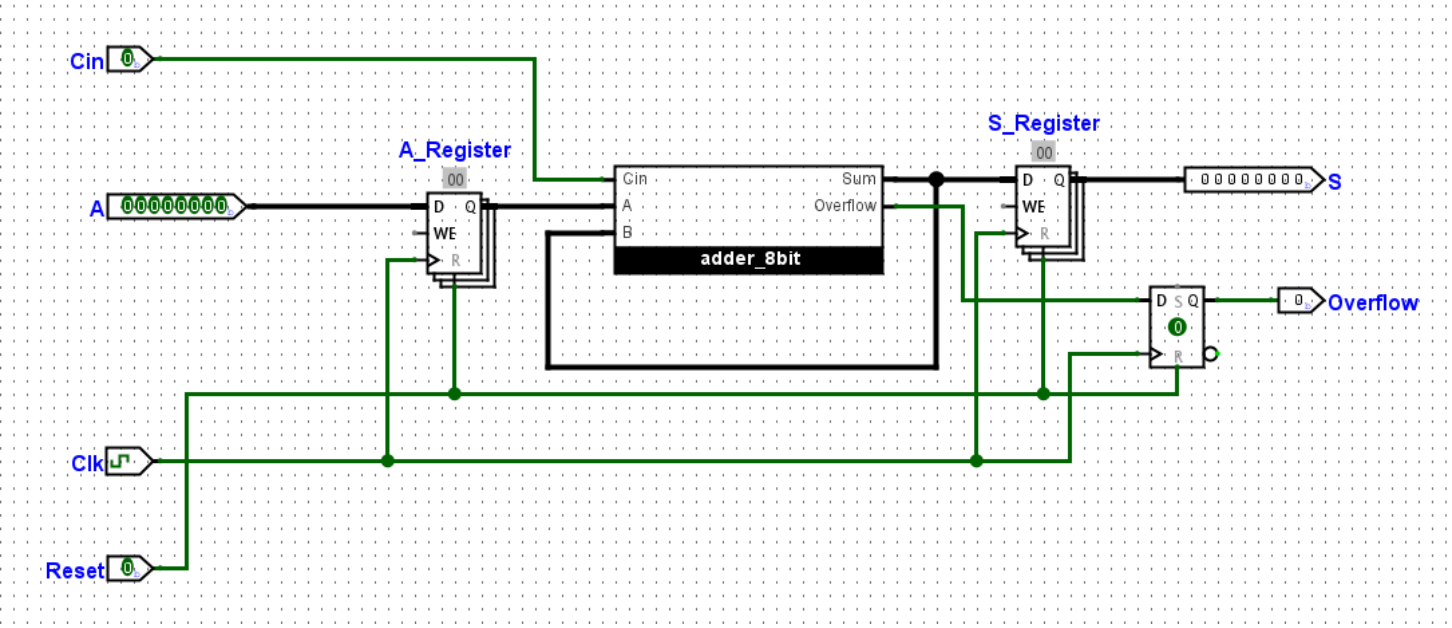
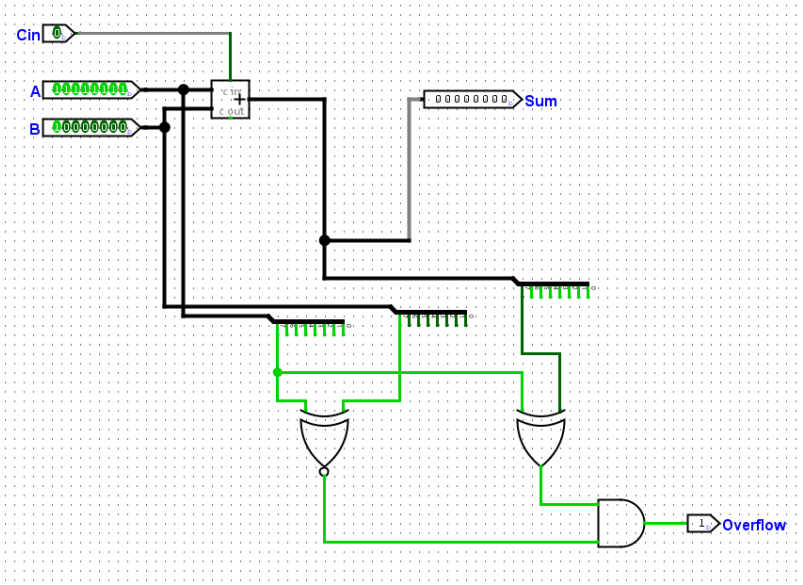
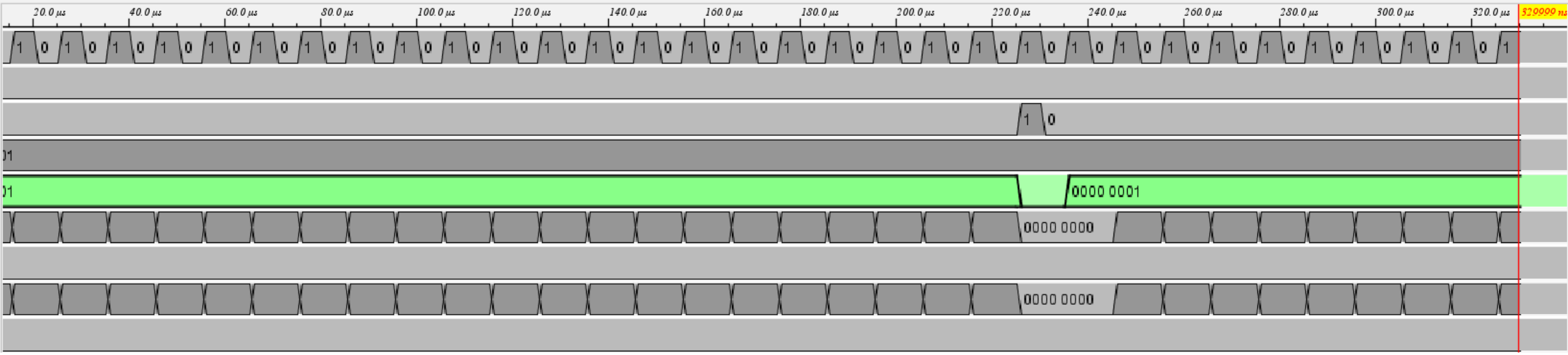


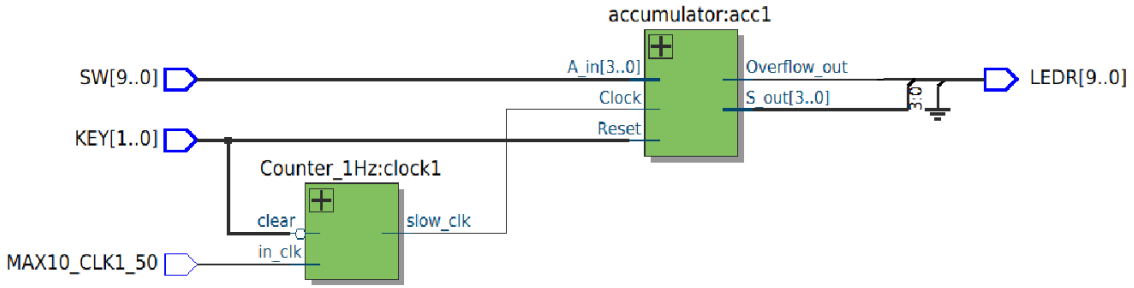
Figure 2: An eight-bit accumulator circuit.

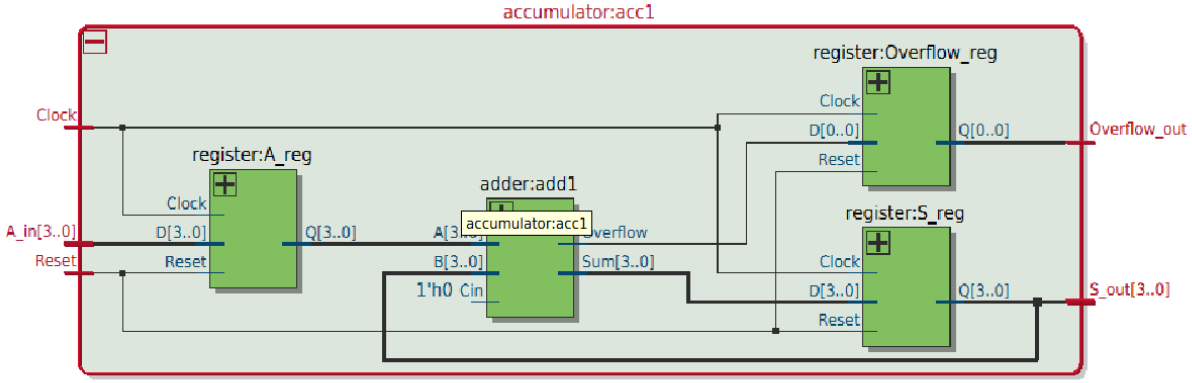

Cin
A
A_Register
S_Register
adder_8bit
Sum
Overflow
S
Overflow
Clk
Reset

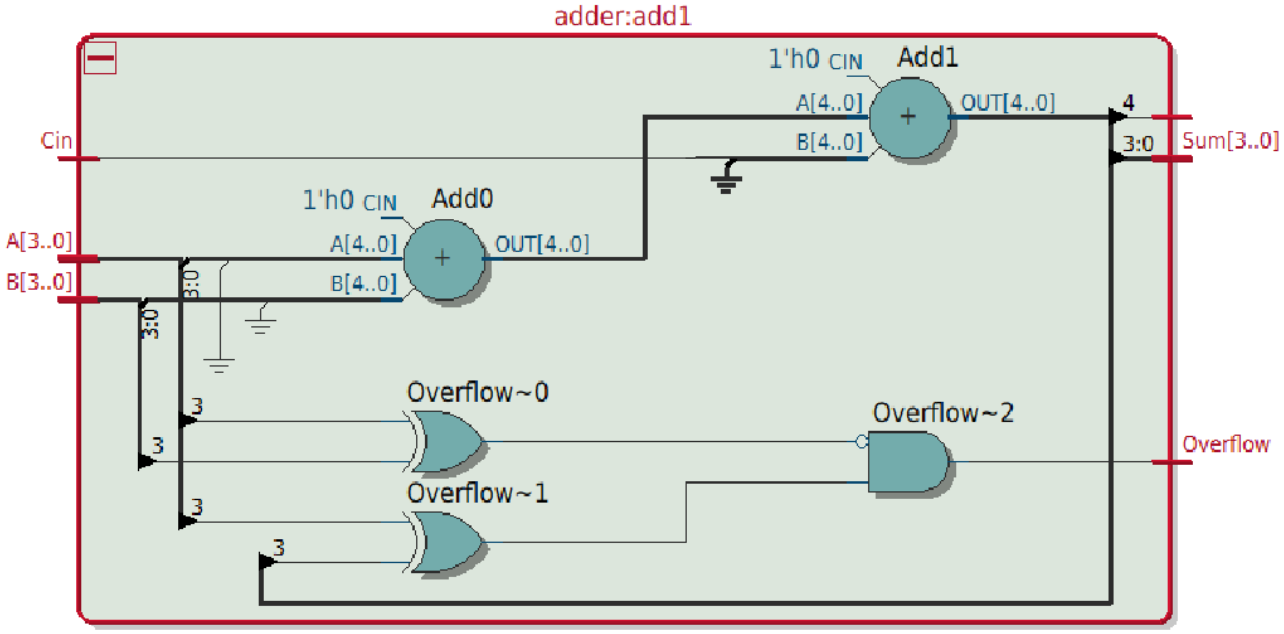

Cin
A
B
Sum
Overflow

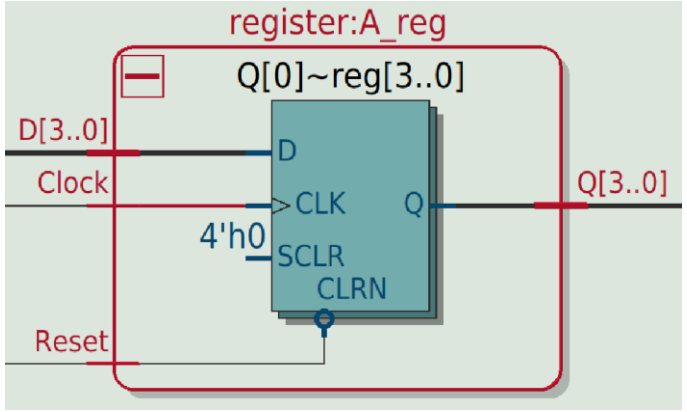
Adder8Bit


20.0 µs
40.0 µs
60.0 µs
80.0 µs
100.0 µs
120.0 µs
140.0 µs
160.0 µs
180.0 µs
200.0 µs
220.0 µs
240.0 µs
260.0 µs
280.0 µs
300.0 µs
320.0 µs
0000 0001
0000 0000
0000 0000


accumulator:acc1
SW[9..0]
KEY[1..0]
MAX10_CLK1_50
A_in[3..0]
Clock
Reset
Overflow_out
S_out[3..0]
3:0
LEDR[9..0]
Counter_1Hz:clock1
clear
in_clk
slow_clk


accumulator:acc1
Clock
register:A_reg
A_in[3..0]
Reset
D[3..0]
Q[3..0]
adder:add1
accumulator:acc1
B[3..0]
1'h0 Cin
Sum[3..0]
register:Overflow_reg
D[0..0]
Q[0..0]
Overflow_out
register:S_reg
S_out[3..0]


adder:add1
Cin
A[3..0]
B[3..0]
1'h0 CIN
Add0
A[4..0]
B[4..0]
OUT[4..0]
Add1
3:0
Sum[3..0]
Overflow~0
Overflow~1
Overflow~2
Overflow


register:A_reg
Q[0]~reg[3..0]
D[3..0]
Clock
D
CLK
Q
Q[3..0]
4'h0
SCLR
CLRN
Reset

```
module adder #(
    parameter N = 4
)(
    input  signed [N-1:0] A,
    input  signed [N-1:0] B,
    input                 Cin,

    output signed [N-1:0] Sum,
    output                Overflow,
    output                Cout
);

    assign {Cout, Sum} = A + B + Cin;

    assign Overflow = ~(A[N-1] ^ B[N-1]) &
                      (A[N-1] ^ Sum[N-1]);

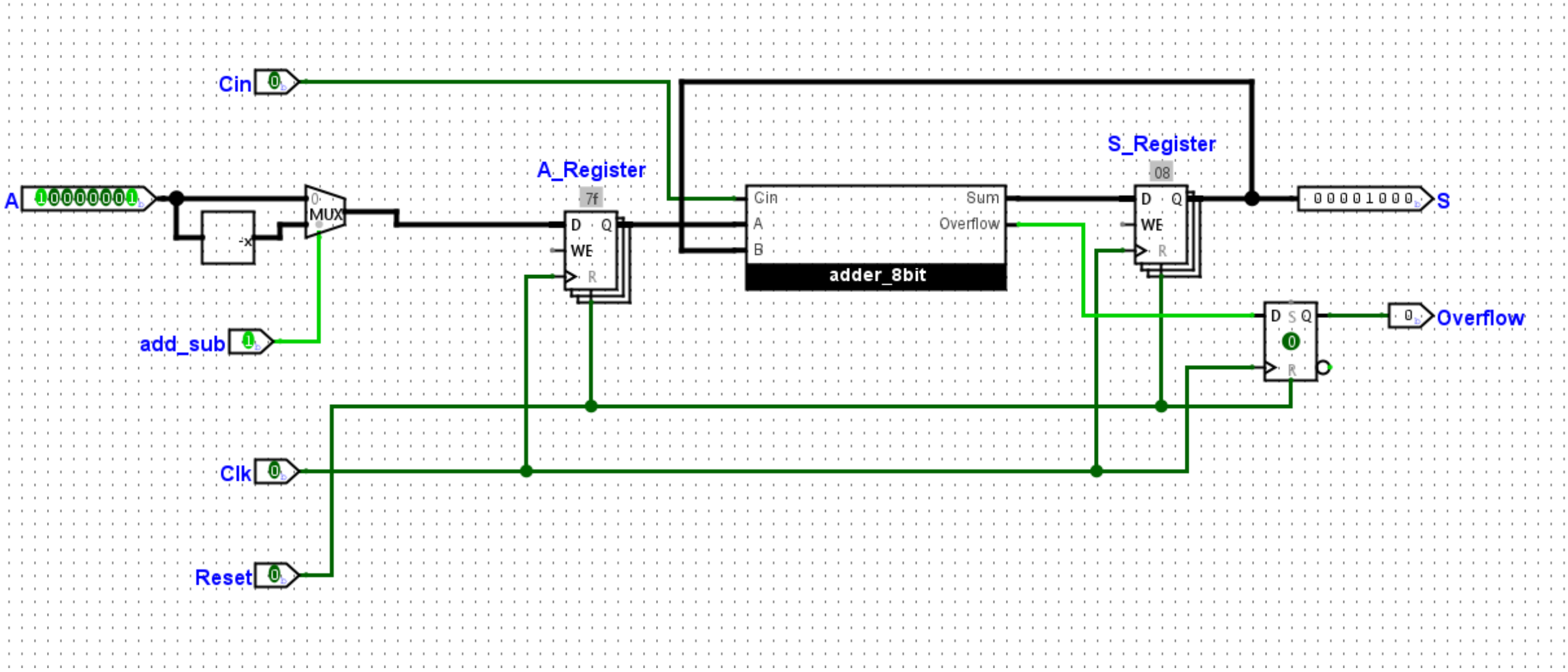
endmodule
```

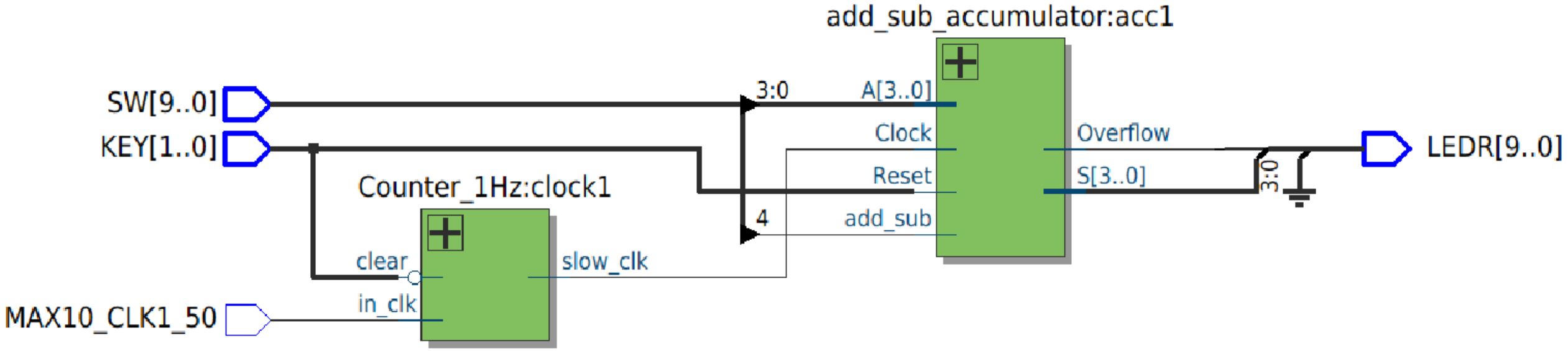
```
module register #(
    parameter N = 4
)(
    input              Clock,
    input              Reset,
    input      [N-1:0] D,
    output reg [N-1:0] Q
);

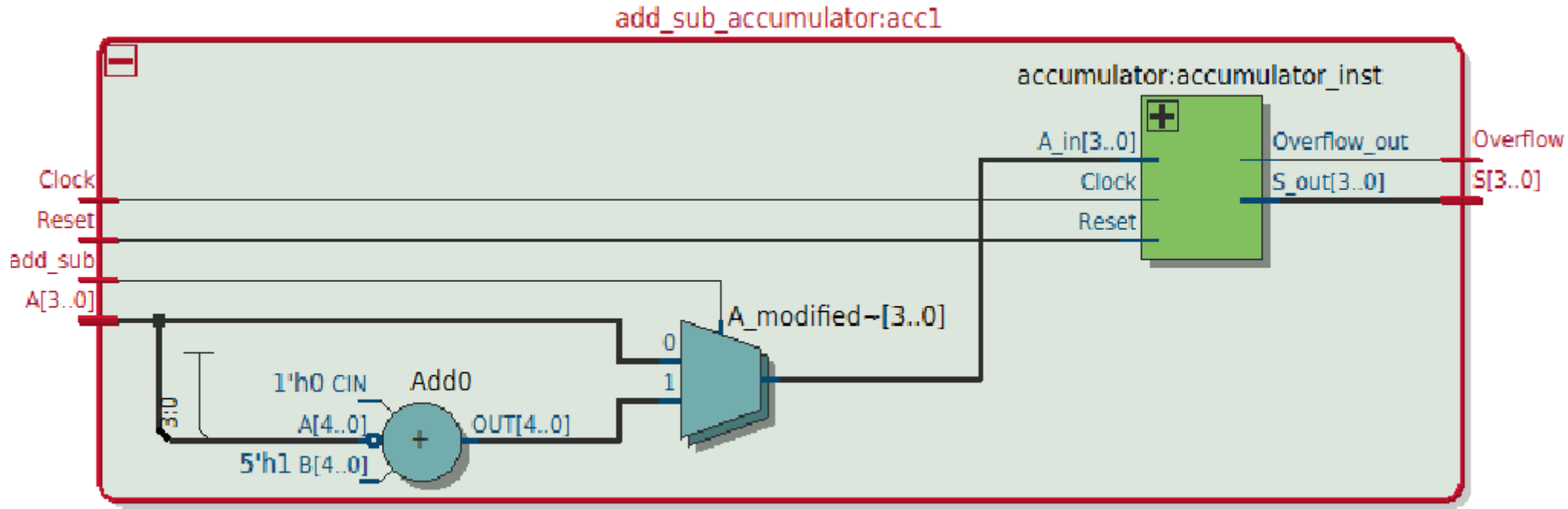
    always @(posedge Clock or negedge Reset) begin
        if (!Reset)
            Q <= {N{1'b0}};
        else
            Q <= D;
    end

endmodule
```

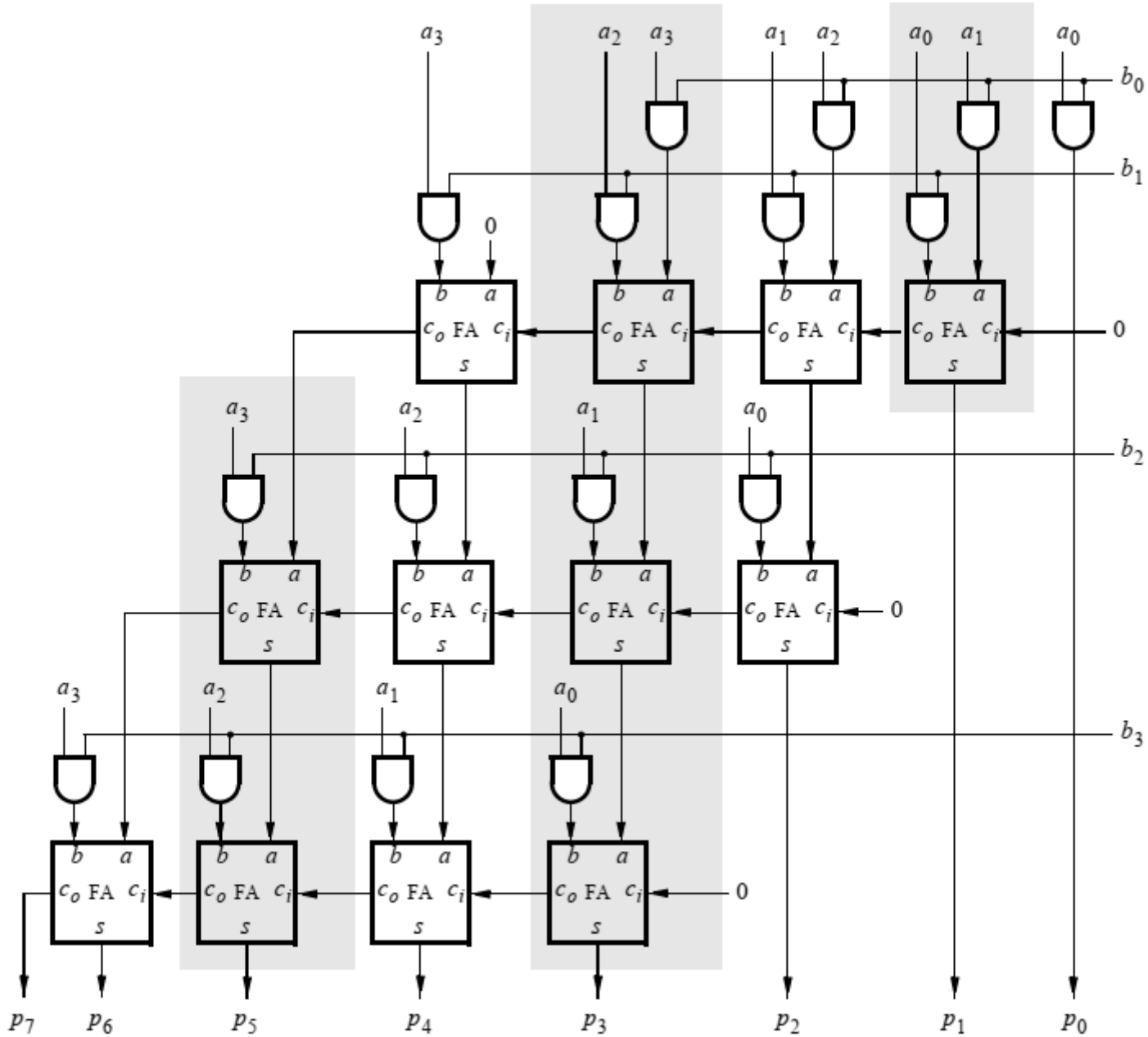
Part II


Cin
A
MUX
-x
add_sub
A_Register
7f
D Q
WE
R
Cin
A
B
Sum
Overflow
adder_8bit
S_Register
08
D Q
WE
R
00001000
S
D S Q
0
R
0
Overflow
Clk
Reset

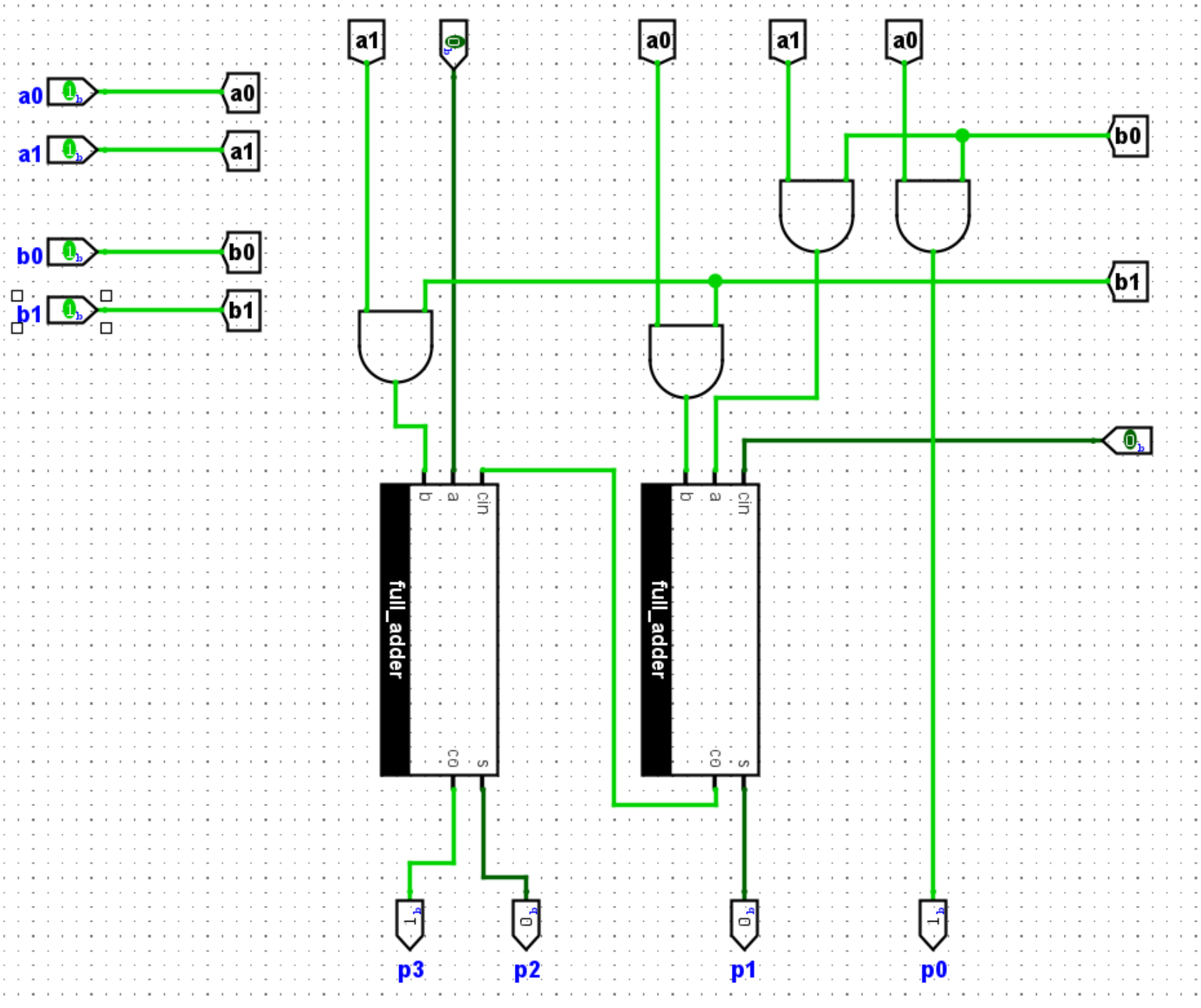

add_sub_accumulator:acc1
SW[9..0]
KEY[1..0]
MAX10_CLK1_50
Counter_1Hz:clock1
clear
in_clk
slow_clk
3:0
4
A[3..0]
Clock
Reset
add_sub
Overflow
S[3..0]
3:0
LEDR[9..0]


add_sub_accumulator:acc1
accumulator:accumulator_inst
Clock
Reset
add_sub
A[3..0]
A_in[3..0]
Clock
Reset
Overflow_out
S_out[3..0]
Overflow
S[3..0]
A_modified~[3..0]
0
1
3:0
1'h0 CIN
Add0
A[4..0]
5'h1 B[4..0]
OUT[4..0]

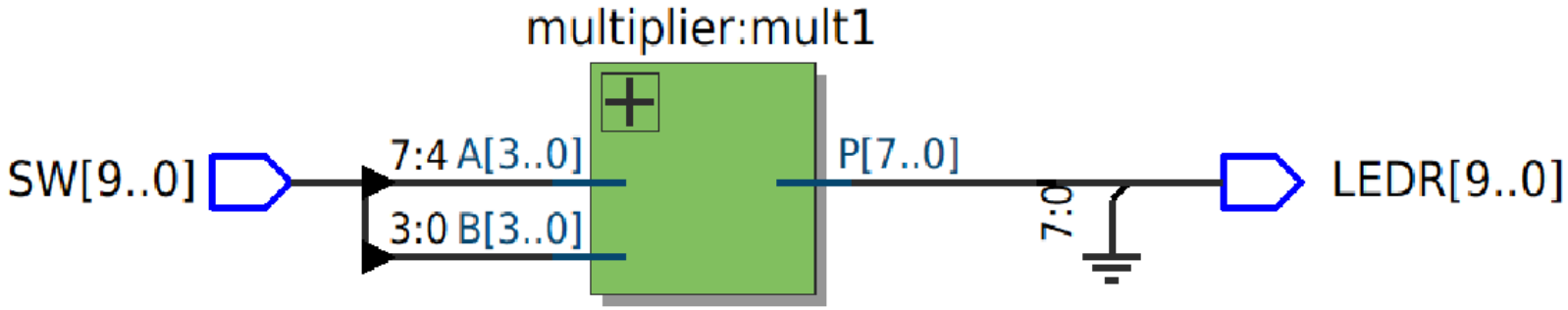
Part III

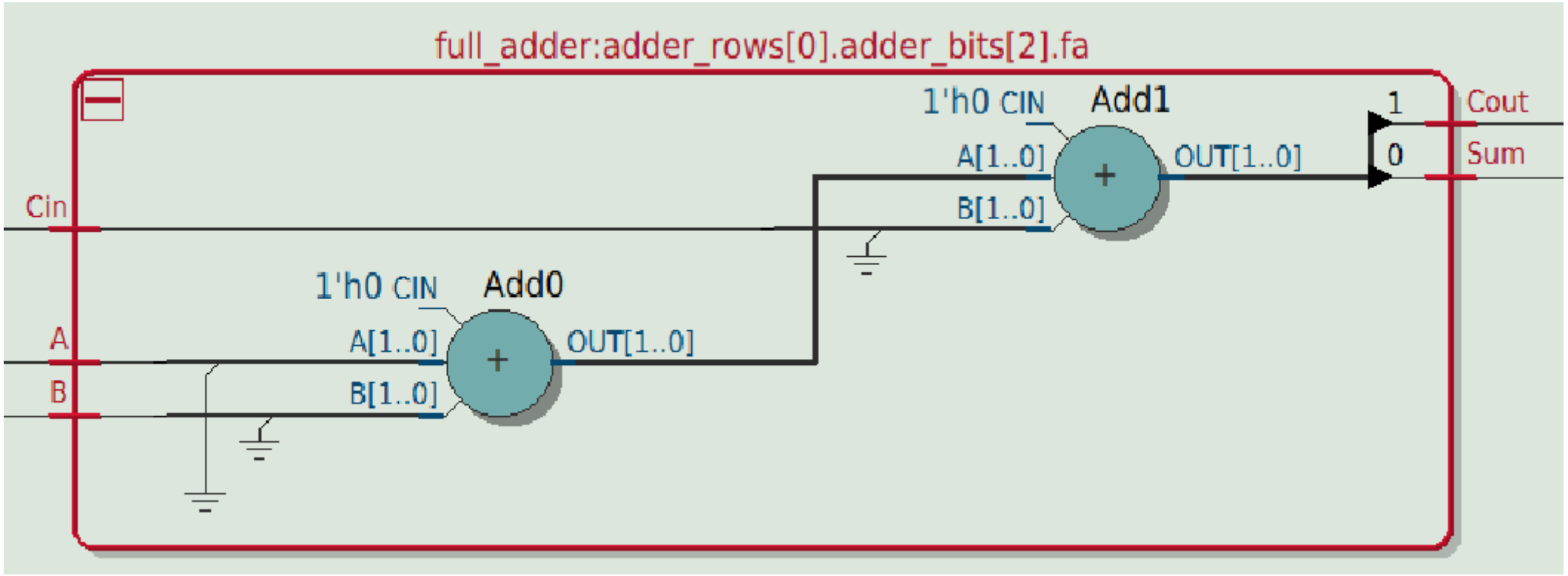

a_3 a_2 a_3 a_1 a_2 a_0 a_1 a_0
b_0
b_1
0
b a
c_o FA c_i
s
0
a_3 a_2 a_1 a_0
b_2
0
a_3 a_2 a_1 a_0
b_3
0
p_7 p_6 p_5 p_4 p_3 p_2 p_1 p_0

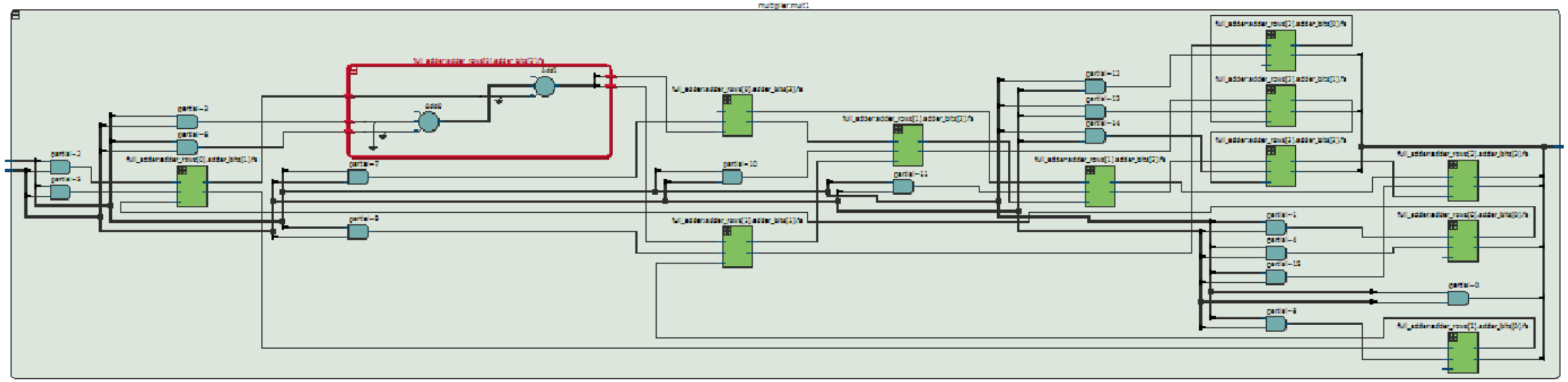
2 bit Multiplier



4 bit Multiplier







```
module multiplier #(
   parameter N = 4
)(
   input  [N-1:0]   A,
   input  [N-1:0]   B,
   output [2*N-1:0] P
);

   wire [N-1:0] partial [0:N-1];
   wire [N:0]   stage   [0:N-1];
   wire [N:0]   carry   [0:N-2];

   genvar r, c;

   generate
      // N partial-product rows
      for (r = 0; r < N; r = r + 1) begin : partial_rows
         assign partial[r] = A & {N{B[r]}};
      end
   endgenerate

   assign stage[0] = {1'b0, partial[0]};
      assign P[0] = stage[0][0];

   generate
      // N-1 rows of N full adders
      for (r = 0; r < N-1; r = r + 1) begin : adder_rows

         assign carry[r][0] = 1'b0;

         for (c = 0; c < N; c = c + 1) begin : adder_bits

            full_adder fa (
               .A(stage[r][c+1]),
               .B(partial[r+1][c]),
               .Cin(carry[r][c]),
               .Sum(stage[r+1][c]),
               .Cout(carry[r][c+1])
            );

         end

   assign stage[r+1][N] = carry[r][N];
         assign P[r+1] = stage[r+1][0];

      end
   endgenerate

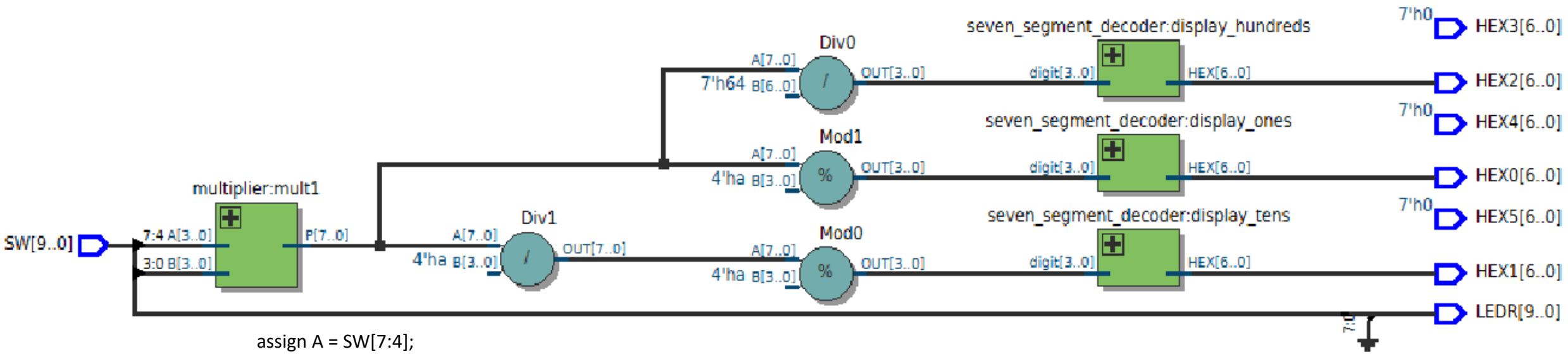
   assign P[2*N-1:N] = stage[N-1][N:1];

endmodule
```

```
module full_adder (
   input  A,
   input  B,
   input  Cin,
   output Sum,
   output Cout
);

   assign {Cout, Sum} =
      {1'b0, A} + {1'b0, B} + {1'b0, Cin};

endmodule
```




```
module main (
   input  [9:0] SW,
   output [9:0] LEDR,

   output [6:0] HEX0,
   output [6:0] HEX1,
   output [6:0] HEX2,
   output [6:0] HEX3,
   output [6:0] HEX4,
   output [6:0] HEX5
);

   wire [3:0] A;
   wire [3:0] B;
   wire [7:0] P;
```

```
assign A = SW[7:4];
assign B = SW[3:0];

assign LEDR[7:4] = A;
assign LEDR[3:0] = B;
assign LEDR[9:8] = 2'b00;

multiplier #(
   .N(4)
) mult1 (
   .A(A),
   .B(B),
   .P(P)
);

wire [3:0] hundreds;
wire [3:0] tens;
wire [3:0] ones;
```

```
assign hundreds = P / 8'd100;
assign tens     = (P / 8'd10) % 8'd10;
assign ones     = P % 8'd10;

seven_segment_decoder display_hundreds (
.digit(hundreds),
.HEX(HEX2)
);

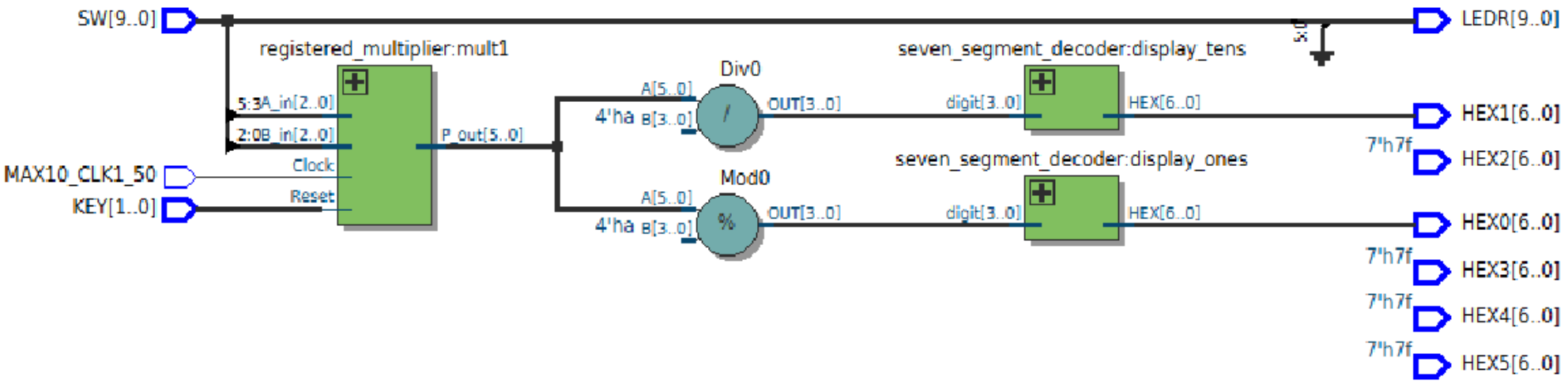
seven_segment_decoder display_tens (
.digit(tens),
.HEX(HEX1)
);
```

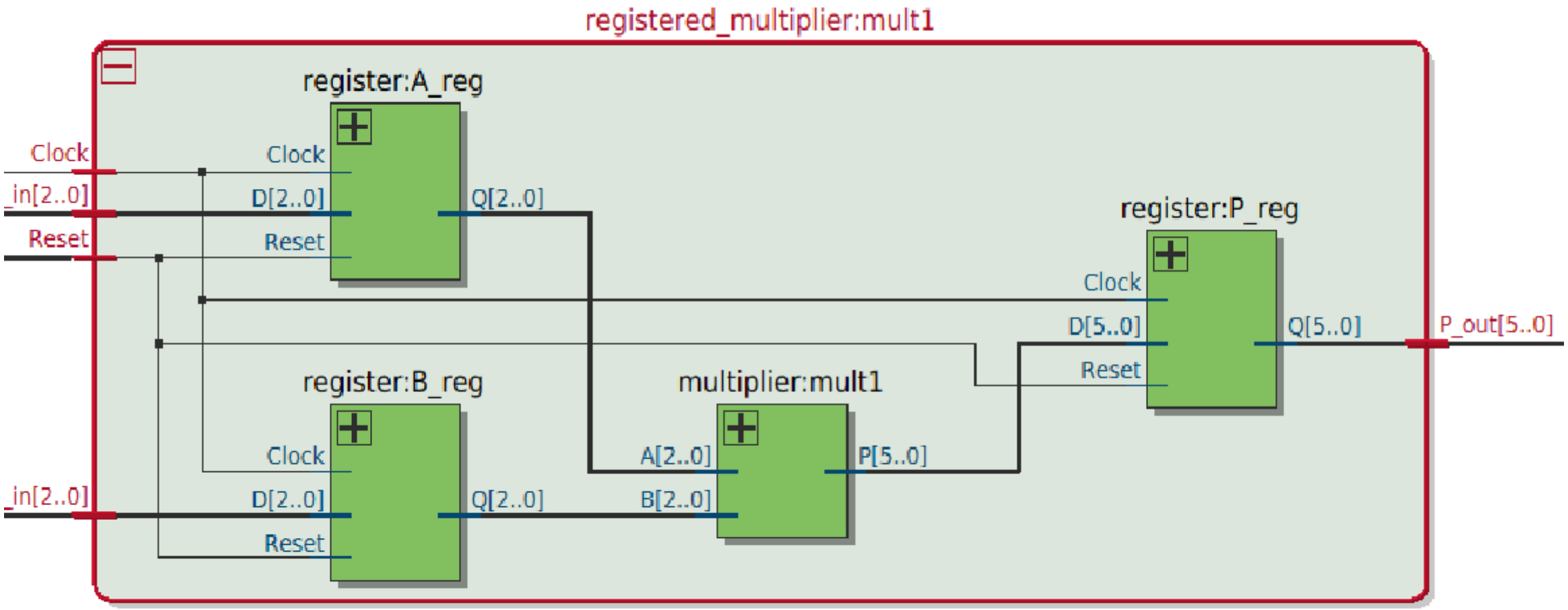
```
seven_segment_decoder display_ones (
 .digit(ones),
.HEX(HEX0)
);

endmodule
```

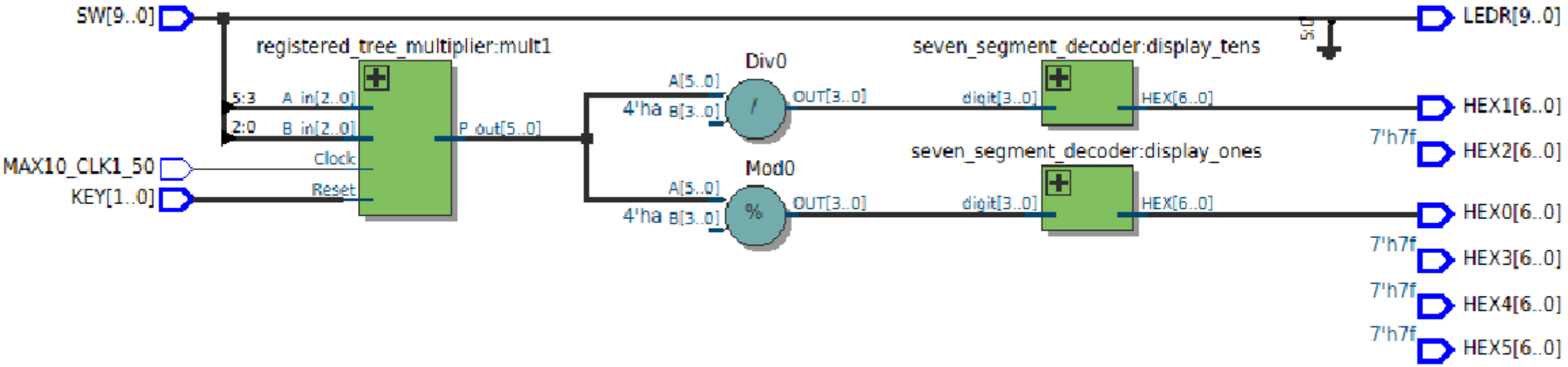
Part IV

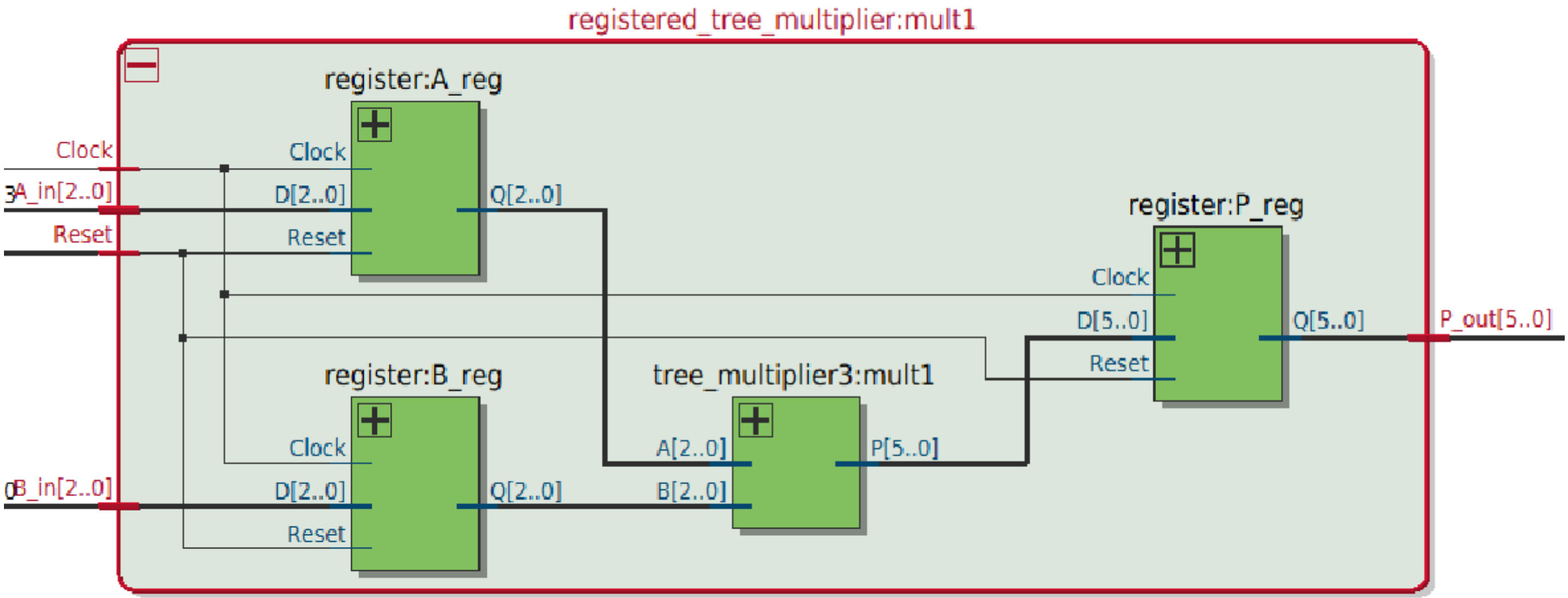
Flow Status	Successful - Wed Oct 7 21:54:18 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	main
Top-level Entity Name	main
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	95 / 49,760 (< 1 %)
Total registers	12
Total pins	65 / 360 (18 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

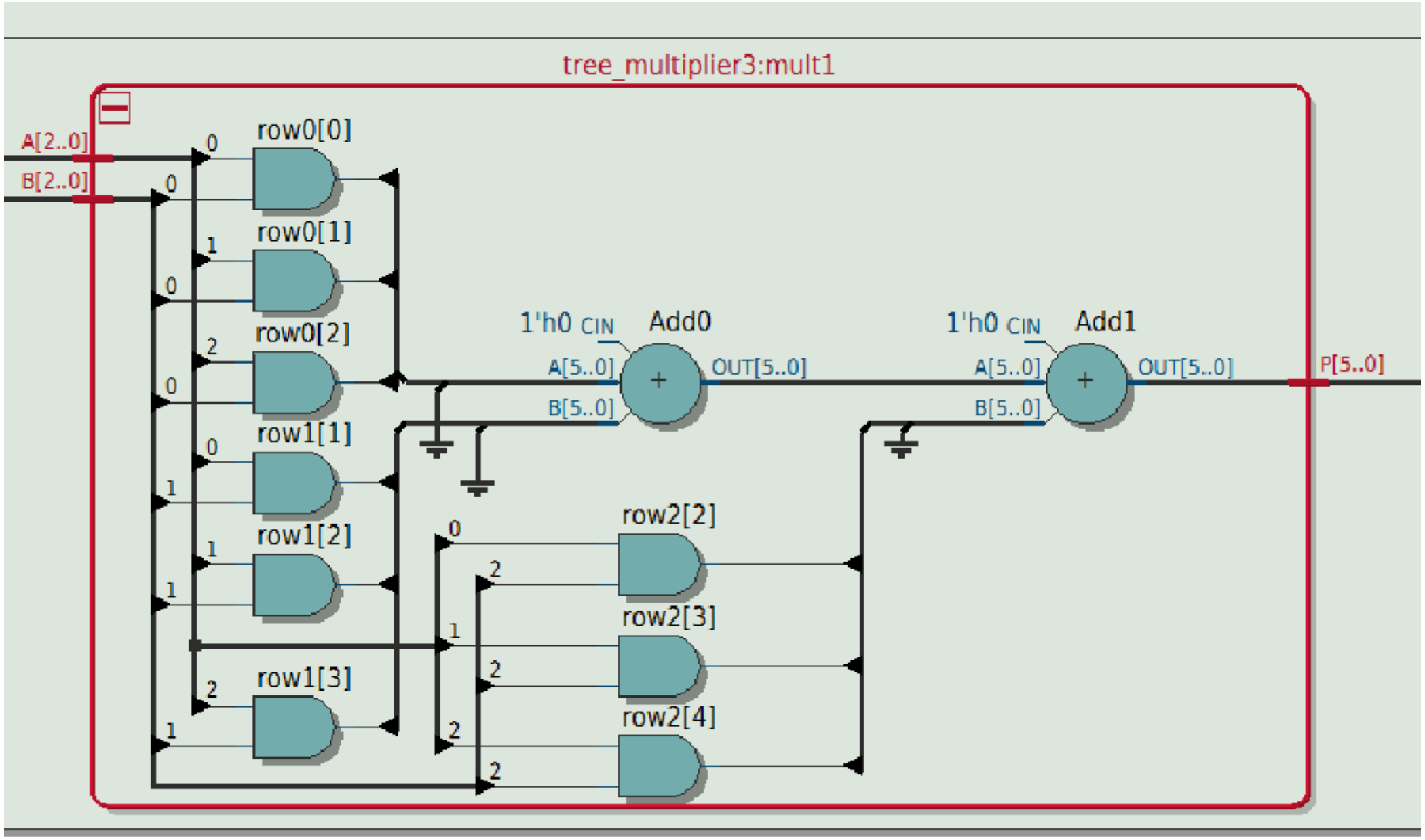



registered_multiplier:mult1
register:A_reg
Clock
D[2..0]
Reset
Q[2..0]
Clock
_in[2..0]
Reset
register:P_reg
Clock
D[5..0]
Reset
Q[5..0]
P_out[5..0]
register:B_reg
Clock
D[2..0]
Reset
Q[2..0]
_in[2..0]
multiplier:mult1
A[2..0]
B[2..0]
P[5..0]

Part V


SW[9..0]
LEDR[9..0]
5:0
registered_tree_multiplier:mult1
5:3 A_in[2..0]
2:0 B_in[2..0]
P_out[5..0]
Clock
Reset
MAX10_CLK1_50
KEY[1..0]
Div0
A[5..0]
4'ha B[3..0]
/
OUT[3..0]
Mod0
A[5..0]
4'ha B[3..0]
%
OUT[3..0]
seven_segment_decoder:display_tens
digit[3..0]
HEX[6..0]
seven_segment_decoder:display_ones
digit[3..0]
HEX[6..0]
HEX1[6..0]
7'h7f
HEX2[6..0]
HEX0[6..0]
7'h7f
HEX3[6..0]
7'h7f
HEX4[6..0]
7'h7f
HEX5[6..0]


registered_tree_multiplier:mult1
register:A_reg
Clock
Clock
A_in[2..0]
D[2..0]
Q[2..0]
Reset
Reset
register:P_reg
Clock
D[5..0]
Q[5..0]
P_out[5..0]
Reset
register:B_reg
tree_multiplier3:mult1
Clock
A[2..0]
P[5..0]
B_in[2..0]
D[2..0]
Q[2..0]
B[2..0]
Reset


tree_multiplier3:mult1
A[2..0]
B[2..0]
row0[0]
row0[1]
row0[2]
row1[1]
row1[2]
row1[3]
row2[2]
row2[3]
row2[4]
1'h0 CIN
Add0
A[5..0]
B[5..0]
OUT[5..0]
1'h0 CIN
Add1
A[5..0]
B[5..0]
OUT[5..0]
P[5..0]